

Drone-Based 3D Reconstruction

Research Execution Guide for Claude Code

System: RTX 4050 (6GB VRAM) | 16GB RAM | Linux

Environment: conda env 'drone3d' at ~/projects/drone3d

Research Direction	Semi-supervised adaptation of feed-forward 3D reconstruction models for drone imagery
Foundation Models	DUST3R, MAST3R, Spann3R, Point3R, MonST3R, St4RTrack, CUT3R
Key Insight	All models are SUPERVISED for 3D (CroCo backbone is self-supervised). Reducing supervision for d
Novelty Target	Self/semi-supervised fine-tuning using photometric consistency, removing need for ground truth 3D da
Claude Code Model	Sonnet for implementation. Opus only for loss function design and paper writing.

Table of Contents

1. Phase 1: Environment Verification and Baseline Test
2. Phase 2: Supervised Baselines on Real Aerial Data
3. Phase 3: Quantitative Evaluation Framework
4. Phase 4: Model Comparison Study (DUS3R vs MAS3R vs Aerial)
5. Phase 5: Your Own Drone Video Pipeline
6. Phase 6: Self-Supervised Adaptation (Novel Research)
7. Phase 7: Synthetic Data Generation for Training
8. Phase 8: Evaluation, Ablation, and Paper Writing
9. Appendix A: Complete Paper Reading List
10. Appendix B: Dataset Catalog
11. Appendix C: Troubleshooting Reference

How to use this guide: Copy each "Claude Code Prompt" block into Claude Code. Run them sequentially. Each phase produces outputs that feed the next. Do not skip phases.

Phase 1: Environment Verification and Baseline Test

Goal: Confirm DUST3R and MAST3R run correctly on your GPU. You already completed initial setup. This phase verifies everything and runs a proper baseline.

Prerequisites (already done)

- conda env 'drone3d' created with Python 3.11
- PyTorch 2.5.1+cu121 installed, GPU detected
- MAST3R cloned at ~/projects/drone3d/mast3r (with DUST3R submodule)
- DUST3R and MAST3R checkpoints in ~/projects/drone3d/checkpoints/
- DUST3R test passed: 274K points, 2.3GB VRAM, 3.3s on NLE tower images

Step 1.1: Run MAST3R on the same images

Claude Code Prompt:

```
In my drone3d project (conda env: drone3d, project at ~/projects/drone3d): Run MAST3R (not DUST3R) on the 7 NLE tower images already at ~/projects/drone3d/data/test_real/ using the checkpoint at ~/projects/drone3d/checkpoints/MAST3R_ViTLarge_BaseDecoder_512_catmlpdpt_metric.pth Settings: 224px resolution, batch_size=1. Run global alignment on CPU with 300 iterations (to save GPU memory - I only have 6GB VRAM). Save the point cloud to ~/projects/drone3d/outputs/mast3r_tower.ply Save the camera poses to ~/projects/drone3d/outputs/mast3r_tower_poses.npz Print: number of 3D points, peak VRAM in MB, inference time in seconds. Use total_memory not total_mem for PyTorch GPU properties. If CUDA OOM, fall back to CPU.
```

Expected output: Similar point count to DUST3R but with metric scale. VRAM should be under 3GB at 224px.

Step 1.2: Visual comparison

Claude Code Prompt:

```
Compare the two point clouds I generated: - ~/projects/drone3d/outputs/dust3r_pointcloud.ply (from DUST3R) - ~/projects/drone3d/outputs/mast3r_tower.ply (from MAST3R) Write a Python script that: 1. Loads both PLY files using open3d 2. Prints stats for each: number of points, bounding box dimensions, point density 3. Computes Chamfer distance between them (sample 50K points from each) 4. Saves a side-by-side visualization as a PNG image to ~/projects/drone3d/outputs/comparison_dust3r_vs_mast3r.png If open3d visualization fails (headless server), use matplotlib 3D scatter plots instead.
```

Phase 2: Supervised Baselines on Real Aerial Data

Goal: Test on proper aerial/drone datasets. The tower images were ground-level. We need to test on actual aerial imagery to understand how these models perform on drone data.

Step 2.1: Download aerial test data

Claude Code Prompt:

```
I need real aerial/drone images for testing 3D reconstruction. Try these sources in order: 1. Clone AerialMegaDepth: git clone https://github.com/kvuong2711/aerial-megadepth.git --depth 1 into ~/projects/drone3d/data/ Check if it has sample images or a small demo dataset. 2. If no images in AerialMegaDepth, download the DroneSplat demo data. Check their GitHub for dataset download links: https://github.com/PanoptiqAI/DroneSplat 3. If neither works, download 15-20 aerial images from the ETH3D outdoor dataset: https://www.eth3d.net/datasets 4. As last resort, use nerfstudio to download Mill19: pip install nerfstudio (inside conda env drone3d) ns-download-data mill19 --save-dir ~/projects/drone3d/data/mill19 Save whatever aerial images you find to ~/projects/drone3d/data/aerial_test/ Print how many images you got and their resolution.
```

Step 2.2: Run MAST3R on aerial data

Claude Code Prompt:

```
Run MAST3R on the aerial images at ~/projects/drone3d/data/aerial_test/ Use checkpoint: ~/projects/drone3d/checkpoints/MASt3R_ViTLarge_BaseDecoder_512_catmlpdpt_metric.pth Settings: 224px, batch_size=1, global alignment on CPU, 300 iterations. If there are more than 15 images, use scene_graph='swin-5' instead of 'complete'. Save point cloud to ~/projects/drone3d/outputs/mast3r_aerial.ply Save poses to ~/projects/drone3d/outputs/mast3r_aerial_poses.npz Print number of points, VRAM, time, and note any visual issues (gaps, floaters, distortions).
```

Step 2.3: Download and test AerialMegaDepth checkpoint (if available)

Claude Code Prompt:

```
Check if the AerialMegaDepth aerial-MASt3R checkpoint is downloadable: 1. Go to https://huggingface.co/kvuong/AerialMegaDepth and check for model files 2. If there's a checkpoint-aerial-mast3r.pth (or similar), download it to ~/projects/drone3d/checkpoints/ 3. If download requires authentication or isn't available, note this and we'll proceed with standard MAST3R If you got the checkpoint, run it on the same aerial images and save to: ~/projects/drone3d/outputs/aerial_mast3r_aerial.ply Compare point counts and any visible quality differences vs standard MAST3R.
```

Phase 3: Quantitative Evaluation Framework

Goal: Build a proper evaluation pipeline with standard metrics. This is the foundation for all future comparisons and ablation studies.

Claude Code Prompt:

```
Create a comprehensive evaluation module at ~/projects/drone3d/scripts/evaluate.py
that: 1. Takes two inputs: predicted point cloud (.ply) and ground truth point cloud
(.ply) or COLMAP sparse reconstruction folder 2. Computes these metrics: - Chamfer
Distance (L1 and L2, bidirectional) - F-Score at thresholds 0.01, 0.05, 0.1 (precision
+ recall + F1) - Accuracy: mean distance from prediction to nearest GT point -
Completeness: mean distance from GT to nearest predicted point - Point cloud
statistics: count, bounding box, density 3. If COLMAP ground truth poses are available
(images.txt format): - Absolute Trajectory Error (ATE) after Procrustes alignment -
Relative Pose Error (RPE) for consecutive frames - Rotation error in degrees,
translation error in meters 4. Saves all metrics to a JSON file 5. Generates a summary
table printed to terminal 6. Works without ground truth too (just prints prediction
stats) Use open3d for point cloud operations. Handle edge cases (empty point clouds,
mismatched scales). Make it importable as a module AND runnable as a script with
argparse. Usage should be: python scripts/evaluate.py --pred outputs/mast3r_aerial.ply
--gt data/aerial_test/gt.ply python scripts/evaluate.py --pred
outputs/mast3r_aerial.ply # no GT, just stats
```

Phase 4: Model Comparison Study

Goal: Systematically compare DUST3R, MAST3R, and (if available) aerial-MASt3R on the same data. This produces your first results table.

Claude Code Prompt:

```
Create a benchmark script at ~/projects/drone3d/scripts/benchmark.py that: 1. Takes a
folder of images as input 2. Runs reconstruction with each available model: - DUST3R
(checkpoint: DUST3R_ViTLarge_BaseDecoder_512_dpt.pth) - MAST3R (checkpoint:
MASt3R_ViTLarge_BaseDecoder_512_catmlpdpt_metric.pth) - AerialMASt3R (checkpoint:
checkpoint-aerial-mast3r.pth, if it exists) 3. For each model, records: - Inference
time - Peak VRAM usage - Number of 3D points generated - Point cloud bounding box
dimensions 4. Saves all point clouds to ~/projects/drone3d/outputs/benchmark/ 5. Runs
evaluate.py on each output (if ground truth available) 6. Produces a comparison table
(printed + saved as CSV) Settings for all: 224px, batch_size=1, CPU alignment, 300
iterations. Handle models that don't exist gracefully (skip with a note). Run it on
~/projects/drone3d/data/test_real/ (NLE tower images).
```

Phase 5: Your Own Drone Video Pipeline

Goal: Process your own drone video or photos through the full pipeline. This is where you start generating your own data for the research.

Step 5.1: Frame extraction from drone video

Claude Code Prompt:

```
Create a drone video processing script at
~/projects/drone3d/scripts/extract_frames.py: 1. Takes a video file (MP4, MOV) as input
2. Extracts frames at configurable FPS (default: 1 fps for drone video) 3. Filters
blurry frames using Laplacian variance (threshold configurable, default 100) 4. Filters
near-duplicate frames using structural similarity (SSIM > 0.95 = duplicate) 5. Extracts
GPS metadata from video if available (DJI SRT files) 6. Saves good frames to an output
directory 7. Prints: total frames, kept frames, rejected (blur), rejected (duplicate)
Usage: python scripts/extract_frames.py --video /path/to/drone_video.mp4 --output
data/my_drone/ --fps 1 --blur-threshold 100
```

Step 5.2: Reconstruct from your data

Claude Code Prompt:

```
Run MAST3R reconstruction on my drone images at ~/projects/drone3d/data/my_drone/ If
there are more than 15 images, select 15 evenly-spaced images. Settings: 224px,
batch_size=1, CPU alignment. Save outputs to ~/projects/drone3d/outputs/my_drone/
Include: point cloud (.ply), poses (.npz), and a matplotlib visualization (.png). Also
run COLMAP on the same images as ground truth comparison: - Install colmap if needed
(apt install colmap) - Run automatic reconstruction - Save COLMAP output to
~/projects/drone3d/outputs/my_drone/colmap/ - Compare MAST3R poses vs COLMAP poses
using evaluate.py
```

Phase 6: Self-Supervised Adaptation (NOVEL RESEARCH)

THIS IS WHERE YOUR ORIGINAL CONTRIBUTION BEGINS. Everything before this was establishing baselines. This phase implements the core research idea: adapting supervised feed-forward models to drone domains using self-supervised signals, reducing dependence on ground truth 3D data.

Research Hypothesis

Feed-forward 3D reconstruction models (DUST3R/MASt3R) trained with full supervision can be adapted to drone-specific domains using primarily photometric self-supervision (multi-view consistency), achieving competitive reconstruction quality while requiring little or no ground truth 3D data for the target domain.

Step 6.1: Implement photometric self-supervision loss

Claude Code Prompt (USE OPIUS FOR THIS ONE):

```
I'm implementing self-supervised fine-tuning for MASt3R on drone imagery. This is my novel research contribution. Create ~/projects/drone3d/scripts/self_supervised_loss.py with: 1. PhotometricLoss class: - Takes predicted pointmaps + confidence from MASt3R - Derives depth maps and camera poses from pointmaps - Warps source view to target view using predicted depth + pose - Computes multi-scale SSIM + L1 photometric loss (like Monodepth2) - Applies auto-masking to handle static pixels - Returns per-pixel loss weighted by MASt3R confidence 2. GeometricConsistencyLoss class: - Forward-backward depth consistency check - If depth predicted from view A at pixel p maps to view B at pixel q, then depth predicted from view B at q should map back near p - Penalizes inconsistent depth predictions 3. SmoothnessLoss class: - Edge-aware depth smoothness (weighted by image gradients) - Prevents depth discontinuities in textureless regions 4. CombinedSelfSupervisedLoss class: - Weighted combination: photometric (1.0) + geometric (0.5) + smoothness (0.001) - Configurable weights Make each loss independently testable with synthetic data. Include a __main__ block that tests each loss with random tensors. This is a research module - include detailed docstrings explaining the mathematical formulation of each loss.
```

Step 6.2: Build the fine-tuning loop

Claude Code Prompt (USE OPIUS):

```
Create ~/projects/drone3d/scripts/finetune_selfsupervised.py: This fine-tunes a pretrained MASt3R model using ONLY self-supervised losses (no ground truth 3D). The key innovation: we freeze the encoder and only train the decoder with LoRA adapters (inspired by Fin3R), using the photometric + geometric consistency losses from self_supervised_loss.py. Architecture: 1. Load pretrained MASt3R checkpoint 2. Freeze encoder completely 3. Add LoRA adapters (rank=8) to decoder attention layers 4. Only train LoRA parameters + prediction heads Training loop: 1. Sample image pairs from drone video frames 2. Run MASt3R forward pass to get pointmaps + confidence 3. Compute self-supervised losses (no GT needed) 4. Backpropagate through LoRA parameters only 5. Log losses, save checkpoints every N steps Settings for 6GB GPU: - 224px resolution - batch_size=1 (accumulate gradients over 4 steps) - Learning rate: 1e-5 with cosine warmup - Max 1000 steps for initial testing - Mixed precision (fp16) to save memory - Gradient checkpointing enabled Save checkpoints to ~/projects/drone3d/checkpoints/selfsupervised/ Log training metrics to ~/projects/drone3d/outputs/training_log.csv Include a --dry-run flag that runs 5 steps and reports memory usage without saving.
```

Step 6.3: Evaluate self-supervised model vs supervised baseline

Claude Code Prompt:

Run a comparison experiment: 1. Take 10 test images from the aerial dataset (held out from training) 2. Run reconstruction with: a) Original MAST3R (supervised baseline) b) Self-supervised fine-tuned MAST3R (from Phase 6.2) c) If available: AerialMegaDepth MAST3R (supervised aerial baseline) 3. For each, compute all metrics from evaluate.py 4. Generate a comparison table and save as CSV + terminal output 5. Create visualization: side-by-side point clouds for each model Save everything to ~/projects/drone3d/outputs/experiment_selfsup_vs_sup/

Phase 7: Synthetic Data Generation

Goal: Generate synthetic drone-view training data with perfect ground truth. This supplements real data and enables controlled experiments.

Claude Code Prompt:

```
Create ~/projects/drone3d/scripts/generate_synthetic.py: Generate synthetic
aerial/drone training pairs using 3D rendering: Option A (lightweight, no external
deps): 1. Use Open3D to render views of 3D meshes from aerial viewpoints 2. Load any
.obj/.ply mesh (or generate simple buildings/terrain procedurally) 3. Place virtual
camera at drone-like positions (50-200m altitude, nadir + oblique) 4. Render RGB images
+ depth maps from each viewpoint 5. Save image pairs with known ground truth depth and
relative poses Option B (if Blender is available): 1. Create simple urban scenes (boxes
as buildings, planes as ground) 2. Script camera paths that mimic drone flight patterns
3. Render with Blender's EEVEE for speed 4. Export RGB + depth + camera poses For
either option, generate: - 100 image pairs minimum - Ground truth depth maps (float32
.npy) - Ground truth relative poses (4x4 matrices) - Ground truth pointmaps (to compare
with MAST3R predictions) Save to ~/projects/drone3d/data/synthetic_drone/ This data
serves two purposes: 1. Evaluate MAST3R with perfect ground truth 2. Use as supervised
signal in semi-supervised training (Phase 6)
```

Phase 8: Evaluation, Ablation, and Paper Writing

8.1 Ablation experiments

Claude Code Prompt:

```
Create ~/projects/drone3d/scripts/ablation.py: Run ablation study on the self-supervised fine-tuning approach. Test these variants: 1. Photometric loss only (no geometric consistency, no smoothness) 2. Photometric + geometric consistency (no smoothness) 3. Full loss (photometric + geometric + smoothness) 4. Different LoRA ranks: 4, 8, 16 5. Frozen encoder vs unfrozen last 2 encoder layers 6. 100 vs 500 vs 1000 training steps For each variant: - Train on the same drone data - Evaluate on held-out test set - Record: Chamfer distance, F-score, ATE, training time, VRAM Save results to ~/projects/drone3d/outputs/ablation/ Generate a LaTeX-formatted table for the paper.
```

8.2 Generate paper figures

Claude Code Prompt:

```
Create publication-quality figures for a research paper: 1. Architecture diagram: Show the fine-tuning setup (frozen encoder, LoRA decoders, self-supervised losses) 2. Qualitative comparison: Grid of images showing input, DUST3R output, MAST3R output, our method output 3. Metric plots: Bar charts comparing methods across datasets 4. Loss curves: Training loss over steps for different configurations 5. Failure case analysis: Examples where our method fails vs succeeds Save all figures as both PNG (300dpi) and PDF (vector) to ~/projects/drone3d/outputs/figures/ Use matplotlib with publication style (no grid, serif fonts, proper axis labels).
```

Appendix A: Paper Reading List (Priority Order)

Tier 1: Must-read before starting (understand the foundation)

Paper	Link	Why
CroCo (NeurIPS 2022)	arxiv.org/abs/2210.10716	Self-supervised backbone for all models
DUST3R (CVPR 2024)	arxiv.org/abs/2312.14132	Foundation architecture - pointmaps
MASt3R (ECCV 2024)	arxiv.org/abs/2406.09756	Metric scale + matching
Monodepth2 (ICCV 2019)	arxiv.org/abs/1806.01260	Core self-supervised depth method

Tier 2: Read during Phase 2-4 (extensions and drone-specific)

Paper	Link	Why
Spann3R (3DV 2025)	arxiv.org/abs/2408.16061	Spatial memory for streaming
Point3R (NeurIPS 2025)	arxiv.org/abs/2507.02863	Pointer memory, drift reduction
MonST3R (ICLR 2025)	monst3r-project.github.io	Dynamic scene handling
St4RTrack (ICCV 2025)	st4rtrack.github.io	Joint reconstruction + tracking
CUT3R (2025)	cut3r.github.io	Recurrent state, long sequences
AerialMegaDepth (CVPR 2025)	arxiv.org/abs/2504.13157	Aerial fine-tuning of DUST3R
DroneSplat (CVPR 2025)	arxiv.org/abs/2503.16964	3DGS for drone imagery
Fin3R (2025)	arxiv.org/abs/2511.22429	LoRA fine-tuning for DUST3R models

Tier 3: Read during Phase 6 (self-supervised approaches)

Paper	Link/Note	Why
SC-DepthV3 (TPAMI 2023)	arxiv.org/abs/2211.03660	Scale-consistent self-supervised depth
FlowMap (3DV 2025)	arxiv.org/abs/2404.15259	Differentiable pose + depth from video
COLMAP-Free 3DGS (CVPR 2024)	arxiv.org/abs/2312.07504	3DGS without COLMAP
E-RayZer (2025)	Self-supervised spatial pre-training	Self-supervised pose estimation
NAS3R	Self-supervised Gaussian regression	Photometric loss for 3DGS + poses
Self-Supervised Depth+Ego (2021)	arxiv.org/abs/2108.01980	Geometric constraints for drone video

Appendix B: Dataset Catalog

Dataset	Type	Size	Ground Truth	Best For
Mill19	Real drone	1940 imgs, 4608x3456	COLMAP poses	Primary benchmark
AerialMegaDepth	Real+synthetic	132K imgs, 137 scenes	Depth + poses	Aerial fine-tuning
DroneSplat	Real drone	24 sequences	Dynamic masks	Dynamic scene testing
UAVD4L	Real UAV	2.5M sq meters	6-DoF poses	Localization testing
UrbanScene3D	Real drone	128K imgs, 16 scenes	LiDAR + mesh	Large-scale evaluation
WHU-MVS	Synthetic aerial	6.7x2.2 km coverage	16-bit depth maps	Training aerial MVS
MatrixCity	Synthetic (UE5)	67K aerial + 452K street	Perfect GT	Controlled experiments
BlendedMVS	Mixed	17K imgs, 113 scenes	Rendered depth	General pre-training
ETH3D	Real, laser scanned	25 scenes	High-precision laser	Accuracy evaluation
Tanks & Temples	Real, laser scanned	14 scenes	Laser scanner GT	Generalization test

Appendix C: Troubleshooting Reference

Problem	Solution
CUDA OOM at 224px	Use device='cpu' for inference (slower but works). Or reduce to 10 images.
CUDA OOM during alignment	Always run alignment on CPU: <code>global_aligner(output, device='cpu')</code>
total_mem AttributeError	PyTorch 2.5+ uses total_memory not total_mem
torch.cuda.amp.autocast warning	Replace with <code>torch.amp.autocast('cuda')</code> . Warning is harmless.
COLMAP not found	<code>sudo apt install colmap</code>
open3d visualization fails	Headless system. Use matplotlib or save to file instead.
Git clone fails (network)	Try with <code>--depth 1</code> . Or download ZIP from GitHub.
Mill19 download 404	Use nerfstudio: <code>ns-download-data mill19</code> . Or use AerialMegaDepth instead.
LoRA import error	<code>pip install peft</code> (Hugging Face PEFT library for LoRA)
Gradient checkpointing crash	Disable it: set <code>use_checkpoint=False</code> . Uses more VRAM but stable.
Point cloud has flying points	Increase confidence threshold in extraction. Filter outliers by distance.
MASt3R slower than DUST3R	Expected. MASt3R has additional matching head. Same architecture otherwise.
Self-supervised loss NaN	Reduce learning rate. Check for zero-division in photometric loss. Clamp depths.

Critical 6GB VRAM Rules (apply to every Claude Code prompt):

- Always use 224px image resolution, never 512px
- Always `batch_size=1`
- Always run global alignment on CPU, not CUDA
- Maximum 15 images with 'complete' scene graph
- Use `scene_graph='swin-5'` for more than 15 images
- Use `total_memory` not `total_mem` for PyTorch GPU properties
- Enable mixed precision (`torch.float16`) for fine-tuning
- For training: gradient accumulation over 4 steps to simulate larger batch
- If all else fails: use Google Colab (free T4 16GB) for heavy runs